

DOM Demonstration

Student Name: P. Dharani**Experiment No.:** 5-4**Course:** Full Stack Web Development**Course Code:** 23CM4121

1 Title

DOM Demonstration

2 Aim

To explain and demonstrate the Document Object Model (DOM) and show how JavaScript can dynamically change the content, style and attributes of HTML elements.

3 Objective

- To understand the structure and purpose of the DOM.
- To access HTML elements using JavaScript.
- To dynamically change the content of an HTML element.
- To dynamically change the style of an HTML element.
- To modify an HTML attribute using JavaScript.
- To use an event listener to perform DOM changes when a button is clicked.

4 Theory

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects. Every HTML element becomes a node in this tree, and JavaScript can access and modify it.

Elements can be selected with methods such as `getElementById()`. Once selected, an element's content can be changed with `textContent`, its appearance with the `style` property, and its HTML attributes with `setAttribute()`. An event listener runs a function when a particular event occurs. In this demonstration a click event on the button triggers all the DOM changes without refreshing the webpage.

5 Algorithm / Procedure

1. Create an HTML page containing a heading, status text, message, image, attribute status text and a button.
2. Give unique IDs to the elements that will be modified.
3. Select the required elements using `getElementById()`.
4. Attach a click event listener to the button.
5. Change the message content using `textContent`.
6. Change the heading colour and font size using the `style` property.
7. Change the message background colour using the `style` property.
8. Change the image alt attribute using `setAttribute()`.
9. Update the status text, attribute message and button text.
10. Run the page in a browser and verify the original and changed DOM states.

6 Program

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Demonstration</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1 id="heading">DOM Demonstration</h1>
    <p id="status">Status: Original DOM</p>

    <div id="message" class="message">
      Time management means planning how to spend your hours so that
      important work is finished on time.
    </div>

    
    <p id="attributeStatus">Image alt attribute: Original Image</p>

    <button id="changeButton">Change the DOM</button>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #ffffff;
  margin: 0;
  padding: 30px;
}

.container {
  width: 560px;
  padding: 24px;
  border: 1px solid #999999;
}

h1 {
  font-size: 32px;
  color: #000000;
  margin-top: 0;
}

.message {
  padding: 12px;
  border: 1px solid #cccccc;
  margin-bottom: 15px;
  line-height: 1.5;
}

img {
  display: block;
  margin-bottom: 8px;
}

button {
  padding: 8px 16px;
  cursor: pointer;
}
```

script.js

```
// Select the elements that will be changed
const heading = document.getElementById("heading");
const statusText = document.getElementById("status");
const message = document.getElementById("message");
const demoImage = document.getElementById("demoImage");
const attributeStatus = document.getElementById("attributeStatus");
const changeButton = document.getElementById("changeButton");

// Function that applies all the DOM changes
function applyChanges() {
  // 1. Change the content
  message.textContent =
    "Time management: Good time management means planning your day, " +
    "setting priorities and avoiding distractions. Making a daily " +
    "to-do list and using deadlines helps you finish tasks on time " +
    "and reduces stress.";

  // 2. Change the style
  heading.style.color = "#c2185b";
  heading.style.fontSize = "44px";
  message.style.backgroundColor = "#fff3cd";

  // 3. Change the attribute
  demoImage.setAttribute("alt", "Updated Image");

  // 4. Update the status texts and the button
  statusText.textContent = "Status: Content, Style and Attribute Changed";
  attributeStatus.textContent =
    "Image alt attribute: " + demoImage.getAttribute("alt");
  changeButton.textContent = "DOM Changes Completed";
}

// Run the function when the button is clicked
changeButton.addEventListener("click", applyChanges);
```

7 Code Explanation

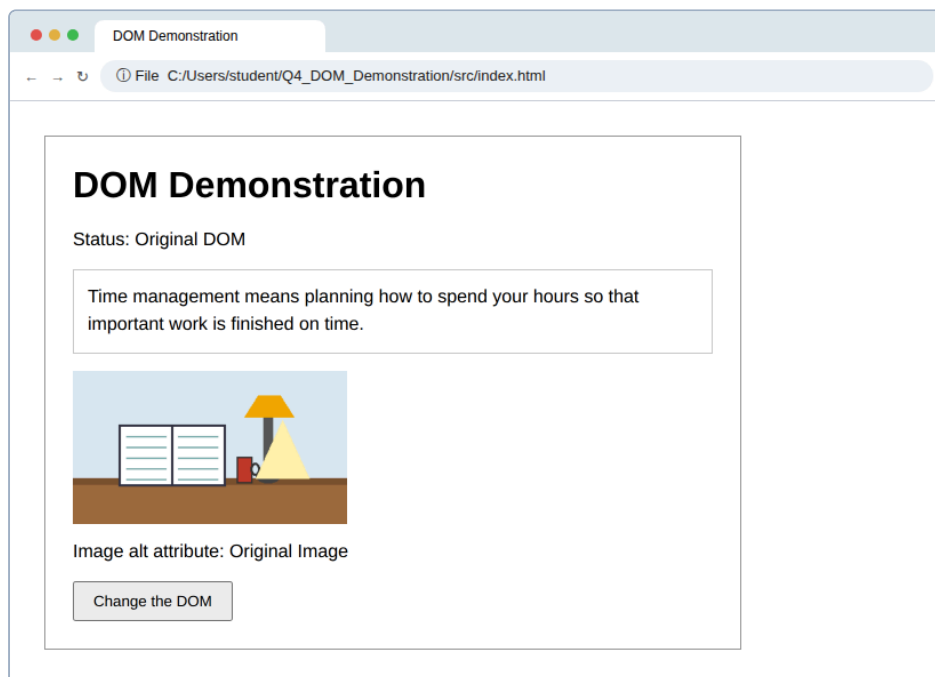
Code	Explanation
<code>document.getElementById("heading")</code>	Selects the heading element by its ID; the same method is used for the status text, message, image, attribute text and button, and each result is stored in a constant.
<code>function applyChanges()</code>	Groups every DOM change in one function so the click handler stays short and easy to read.
<code>message.textContent = "..."</code>	Replaces the text inside the message box with new content.
<code>heading.style.color,</code> <code>heading.style.fontSize</code>	Change the heading colour to crimson (#c2185b) and its font size from 32px to 44px.
<code>message.style.backgroundColor</code>	Gives the message box a light yellow background (#fff3cd).
<code>demoImage.setAttribute("alt", "Updated Image")</code>	Changes the alt attribute of the image from "Original Image" to "Updated Image".
<code>attributeStatus.textContent = ...</code> <code>getAttribute("alt")</code>	Reads the alt value back from the image with <code>getAttribute()</code> and shows it on the page, proving that the attribute really changed.
<code>statusText.textContent,</code> <code>changeButton.textContent</code>	Update the status line and the button label to show that the changes are complete.
<code>changeButton.addEventListener("click",</code> <code>applyChanges)</code>	Registers a click event listener, so all changes run when the button is pressed, without reloading the page.

8 Test Cases and Execution Data

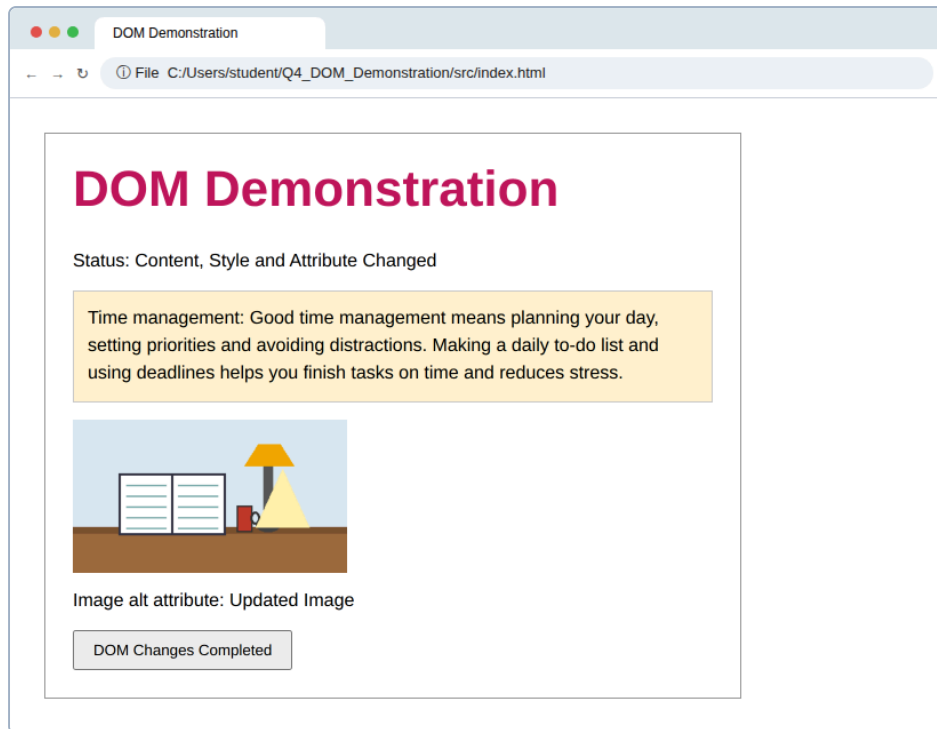
Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Load index.html	Original DOM state is shown: status "Status: Original DOM", black heading (32px), alt "Original Image", button "Change the DOM".	Status text: "Status: Original DOM"; alt attribute: "Original Image"; button: "Change the DOM"; heading colour and size come from style.css.	Pass
TC-02	Click "Change the DOM"	Message text is replaced, status becomes "Status: Content, Style and Attribute Changed".	Status text: "Status: Content, Style and Attribute Changed"; message now begins "Time management: Good time management...".	Pass
TC-03	Check heading and message style after click	Heading colour crimson (#c2185b), size 44px; message background #fff3cd.	Heading colour rgb(194, 24, 91), size 44px; message background rgb(255, 243, 205).	Pass
TC-04	Check image alt attribute after click	Alt attribute changes to "Updated Image".	getAttribute("alt") returned "Updated Image"; page shows "Image alt attribute: Updated Image".	Pass
TC-05	Check button text after click	Button text changes to "DOM Changes Completed".	Button shows "DOM Changes Completed".	Pass
TC-06	Click the button a second time (special case)	No error; the page stays in the same changed state.	No console errors; all values identical to TC-02 to TC-05 (unchanged).	Pass

9 Output

Output 1: Original DOM state (before clicking the button)



Output 2: DOM after JavaScript changes (after clicking the button)



10 Result

The DOM was explained and demonstrated using HTML, CSS and JavaScript. The outputs show the original DOM state and the changes made to the message content, heading colour and size, message background, image alt attribute, status text and button text after the button click.

Submission folder structure

```
Q4_DOM_Demonstration/  
├── src/  
│   ├── index.html  
│   ├── style.css  
│   ├── script.js  
│   └── desk.png  
└── README.md
```